5

Использование пакетов Oracle при разработке Web-приложений

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Рассматриваемые вопросы

- Описание работы пакета `DBMS_OUTPUT`
- Использование пакета `UTL_FILE` для прямого вывода в файлы операционной системы
- Использование пакета `HTP` для генерации простой Web-страницы
- Описание основных возможностей пакета `UTL_MAIL`
- Использование пакета `DBMS_SCHEDULER` для планирование выполнения кода PL/SQL

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Цели урока

На этом уроке вы узнаете, как можно использовать пакеты, поставляемые корпорацией Oracle, и их возможности.

Основное внимание уделяется пакетам, генерирующим выходные данные в текстовом виде и в формате, позволяющем использовать их в Web, обрабатывающим электронную почту и предоставляющим возможности планирования.

Использование пакетов, поставляемых Oracle

Пакеты:

- **Поставляются вместе с сервером Oracle**
- **Расширяют функциональные возможности базы данных**
- **Предоставляют доступ к определенным возможностям SQL, на использование которых в PL/SQL обычно накладываются ограничения.**

Например, пакет `DBMS_OUTPUT` изначально проектировался для отладки программ PL/SQL.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование пакетов, поставляемых Oracle

Пакеты, поставляемые вместе с сервером Oracle

- предоставляют доступ из PL/SQL к определенным возможностям SQL,
- расширяют функциональные возможности базы данных.

Вы можете воспользоваться преимуществами этих пакетов при создании вашего приложения или же просто использовать заложенные в них идеи в ходе создания своих собственных хранимых процедур.

Большинство стандартных пакетов создается в результате выполнения командного файла `catproc.sql`.

`DBMS_OUTPUT` – это один из таких пакетов, с которым в ходе этого курса вы познакомитесь в наибольшей степени. Первоначальные сведения об этом пакете предоставляются в курсе *Введение в Oracle 10g: основы PL/SQL*.

Список некоторых пакетов, поставляемых Oracle

- DBMS_ALERT
- DBMS_LOCK
- DBMS_SESSION
- DBMS_OUTPUT
- HTP
- UTL_FILE
- UTL_MAIL
- DBMS_SCHEDULER

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Список некоторых пакетов, поставляемых Oracle

Список пакетов PL/SQL, поставляемых вместе с базой данных Oracle, возрастает при появлении новой версии. Невозможно охватить в этом курсе все множество пакетов и их функциональные возможности. Дополнительную информацию см. в документе *PL/SQL Packages and Types Reference 10g* (ранее называвшемся *PL/SQL Supplied Packages Reference)*. В этом уроке рассматриваются последние пять пакетов из приводимого списка.

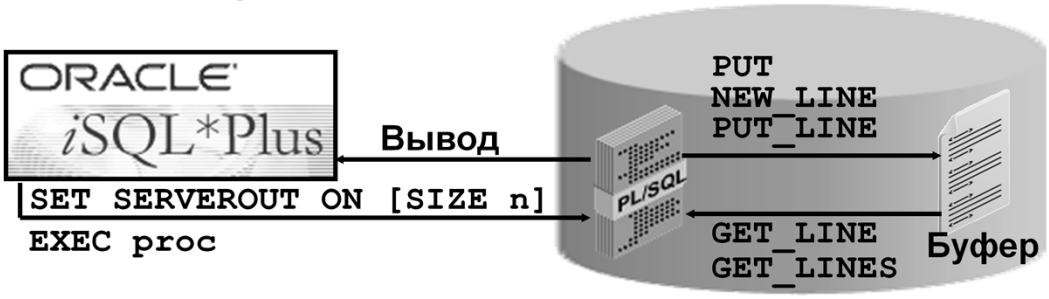
Краткое описание перечисленные пакетов.

- DBMS_ALERT поддерживает асинхронное уведомление о событиях базы данных. Сообщения или *сигналы* (*alerts*) посылаются после команды COMMIT.
- DBMS_LOCK используется для запроса, преобразования и освобождения блокировок с помощью служб Oracle Lock Management.
- DBMS_SESSION позволяет использовать в программе команду ALTER SESSION языка SQL и другие команды уровня сеанса.
- DBMS_OUTPUT предоставляет возможности отладки и использования буфера для текстовых данных.
- HTP пишет данные с тегами HTML в буферы базы данных.
- UTL_FILE позволяет читать и писать в файлы операционной системы.
- UTL_MAIL позволяет формировать и посылать сообщение по электронной почте.
- DBMS_SCHEDULER предоставляет возможности планирования и автоматического выполнения блоков PL/SQL, хранимых процедур, внешних процедур и исполняемых модулей.

Как работает пакет DBMS_OUTPUT

Пакет DBMS_OUTPUT позволяет передавать сообщения из хранимых подпрограмм и триггеров.

- **PUT и PUT_LINE помещают текст в буфер.**
- **GET_LINE и GET_LINES читают текст из буфера.**
- **Сообщения не передаются до тех пор, пока отправитель сообщения не завершит выполнение.**
- **Для вывода сообщений в *i*SQL*Plus используйте команду SET SERVEROUTPUT ON .**



Copyright © 2004, Oracle. All rights reserved.

Использование пакета DBMS_OUTPUT

Пакет DBMS_OUTPUT передает текстовые сообщения из блоков PL/SQL в буфер базы данных. Процедуры пакета:

- PUT добавляет текст в текущую строку выходного строкового буфера.
- NEW_LINE помещает в выходной буфер маркер конца строки.
- PUT_LINE объединяет действия процедур PUT и NEW_LINE; ведущие пробелы удаляются;
- GET_LINE выбирает текущую строку из буфера и помещает в переменную процедуры.
- GET_LINES выбирает массив строк в переменную процедуры, позволяющую хранить массив.
- ENABLE/DISABLE включает/отключает вызовы процедур пакета DBMS_OUTPUT.

Размер буфера может быть задан следующим образом:

- В команду SET SERVEROUTPUT ON добавляется опция SIZE n , где n от 2000 (по умолчанию) до 1 000 000 (1 миллион символов).
- В вызове процедуры ENABLE указывается целый параметр от 2 000 до 1 000 000.

Практическое применение

- Можно в целях отладки вывести выходные результаты в окно.
- Можно отслеживать путь выполнения кода функции или процедуры.
- Можно пересылать сообщения между подпрограммами и триггерами.

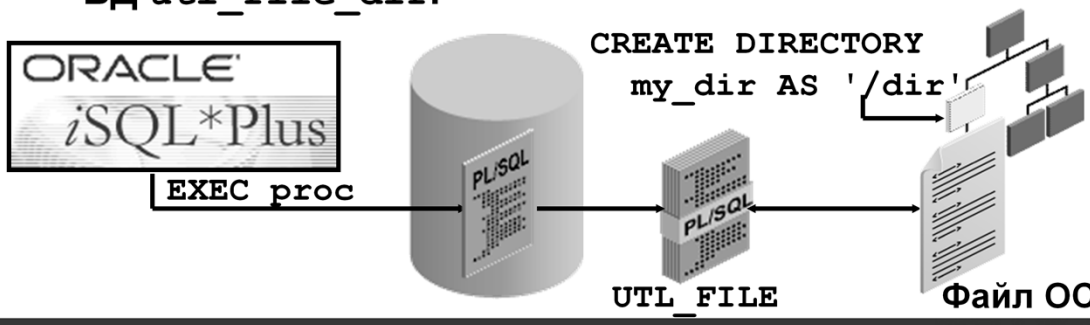
Примечание: нет механизма для сброса (flush) вывода во время выполнения процедуры.

Работа с файлами операционной системы

UTL_FILE расширяет возможности программирования на PL/SQL и позволяет читать и писать в текстовые файлы ОС.

UTL_FILE:

- **Предоставляет ограниченную версию потокового ввода-вывода текстовых файлов ОС**
- **Предоставляет доступ к файлам в директории ОС, определенной по команде CREATE DIRECTORY. Для открытия доступа можно также использовать параметр БД utl_file_dir.**



Copyright © 2004, Oracle. All rights reserved.

Работа с файлами операционной системы

Поставляемый Oracle пакет UTL_FILE используется для доступа к расположенным на сервере базы данных текстовым файлам операционной системы. База данных предоставляет возможности доступа для чтения и записи в конкретные директории ОС, задаваемые следующим образом:

- Команда CREATE DIRECTORY связывает псевдоним с директорией ОС. В базе данных можно предоставить привилегии READ и WRITE для доступа к файлам ОС через псевдоним директории. Например:

```
CREATE DIRECTORY my_dir AS '/temp/my_files';
GRANT READ, WRITE ON my_dir TO public.
```

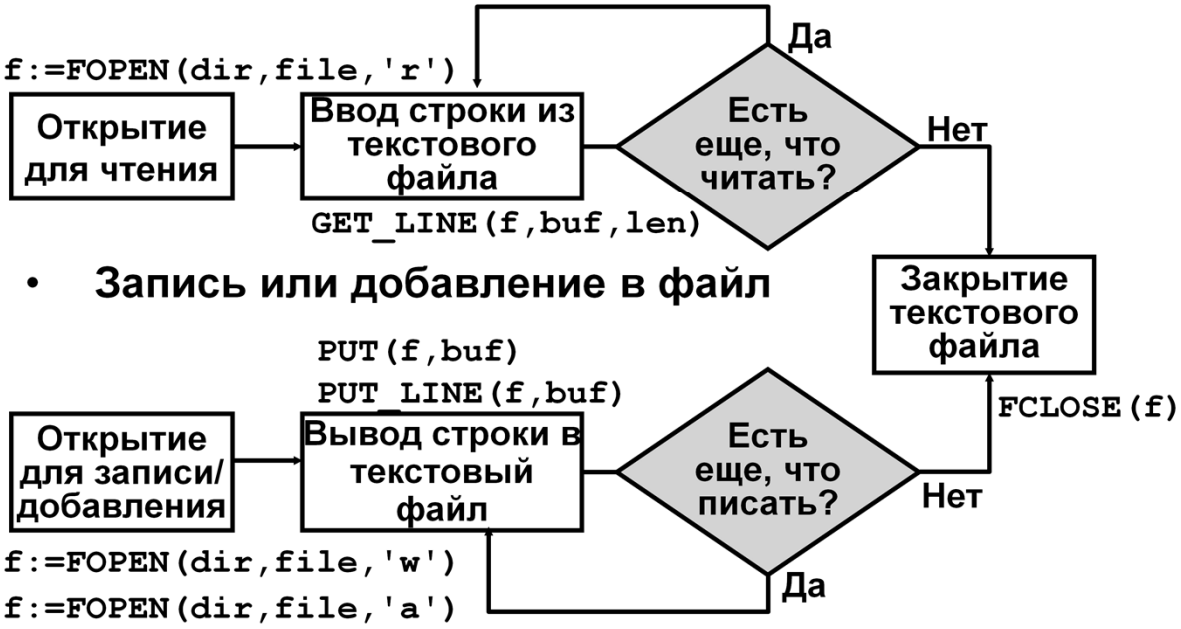
- Пути доступа перечисляются в параметре инициализации БД utl_file_dir.

Предпочтительнее использовать псевдоним директории, созданный по команде CREATE DIRECTORY, так как при этом не требуется перезапускать базу данных. Директории ОС, задаваемые этими двумя методами, должны быть доступны на той же машине, на которой выполняется сервер базы данных. В некоторых ОС имя пути (директории) может зависеть от регистра символов.

Примечание. Пакет DBMS_LOB может использоваться для чтения двоичных файлов операционной системы. DBMS_LOB рассматривается в уроке “Работа с большими объектами”.

Обработка файла с использованием пакета UTL_FILE

- **Чтение файла**



- **Запись или добавление в файл**

Copyright © 2004, Oracle. All rights reserved.

Обработка файла с использованием пакета UTL_FILE

Использование процедур и функций пакета UTL_FILE начинается с открытия файла с помощью функции FOPEN. После этого можно либо читать, либо писать, либо добавлять в файл до полного завершения обработки. Затем файл закрывается с помощью процедуры FCLOSE. Подпрограммы пакета:

- Функция FOPEN открывает файл в заданной директории для ввода/вывода и возвращает *описатель файла* (*file handle*), используемый в последующих операциях ввода/вывода.
- Функция IS_OPEN возвращает логическое значение, показывающее, указывает ли описатель файла на открытый файл. Используйте функцию IS_OPEN перед открытием файла для проверки того, был ли уже до этого открыт этот файл.
- Процедура GET_LINE читает строку текста из файла и помещает ее в задаваемый параметром выходной буфер (максимальная длина входной записи – 1023 байта, если только больший размер не указан в перезагружаемой версии функции FOPEN).
- Процедуры PUT и PUT_LINE пишут текст в открытый файл.
- Процедура PUTF предоставляет возможность форматированного вывода с использованием двух *форматных описателей* (*format specifiers*): %s для подстановки значения в выводимую строку и \n для символа новой строки.
- Процедура NEW_LINE завершает строку в выходном файле.
- Процедура FFLUSH записывает все данные из буфера памяти в файл.
- Процедура FCLOSE закрывает открытый файл.
- Процедура FCLOSE_ALL закрывает все открытые в сеансе файлы.

Исключения пакета UTL_FILE

При использовании подпрограмм пакета UTL_FILE может понадобиться обработать одно из следующих исключений:

- INVALID_PATH
- INVALID_MODE
- INVALID_FILEHANDLE
- INVALID_OPERATION
- READ_ERROR
- WRITE_ERROR
- INTERNAL_ERROR

Другие исключения, не входящие в пакет UTL_FILE:

- NO_DATA_FOUND и VALUE_ERROR

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Исключения пакета UTL_FILE

В пакете UTL_FILE объявлено семь исключений, которые возбуждаются, чтобы показать ошибочные ситуации, возникающие при обработке файлов операционной системы.

- INVALID_PATH; местоположение файла либо имя файла неверно.
- INVALID_MODE; неверное значение параметра OPEN_MODE в функции FOPEN.
- INVALID_FILEHANDLE; неверный описатель файла.
- INVALID_OPERATION; файл не может быть открыт либо надлежащим образом обработан.
- READ_ERROR; возникла ошибка операционной системы в ходе операции чтения.
- WRITE_ERROR; возникла ошибка операционной системы в ходе операции записи.
- INTERNAL_ERROR; неопределенная ошибка возникла в PL/SQL.

Примечание. Этим исключениям всегда должно предшествовать имя пакета. Процедуры пакета UTL_FILE могут также возбудить предопределенные исключения PL/SQL, например, NO_DATA_FOUND или VALUE_ERROR.

Исключение NO_DATA_FOUND возбуждается, когда процедура GET_LINE или GET_LINES пакета UTL_FILE пытается выполнить чтение после достижения конца файла.

Параметры функций FOPEN и IS_OPEN

```
FUNCTION FOPEN (location IN VARCHAR2,
                filename IN VARCHAR2,
                open_mode IN VARCHAR2)
RETURN UTL_FILE.FILE_TYPE;
```

```
FUNCTION IS_OPEN (file IN FILE_TYPE)
RETURN BOOLEAN;
```

Пример:

```
PROCEDURE read(dir VARCHAR2, filename VARCHAR2) IS
  file UTL_FILE.FILE_TYPE;
BEGIN
  IF NOT UTL_FILE.IS_OPEN(file) THEN
    file := UTL_FILE.FOPEN (dir, filename, 'r');
  END IF; ...
END read;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Параметры функций FOPEN и IS_OPEN

- `location` задает имя псевдонима директории, определенного с помощью команды `CREATE DIRECTORY`, либо зависимый от операционной системы путь, указанный в параметре базы данных `utl_file_dir`.
- `filename` задает имя файла, содержащее расширение, без пути доступа.
- `open_mode` определяет строку, в которой показывается, для чего открывается файл; допускаются следующие значения:
  - `'r'` – для чтения текста (с помощьюе `GET_LINE`);
  - `'w'` – для записи текста (`PUT`, `PUT_LINE`, `NEW_LINE`, `PUTF`, `FFLUSH`);
  - `'a'` – для добавления текста (`PUT`, `PUT_LINE`, `NEW_LINE`, `PUTF`, `FFLUSH`).

Возвращаемое значение функции `FOPEN` – описатель файла с типом `UTL_FILE.FILE_TYPE`. Этот описатель должен использоваться в последующих вызовах подпрограмм, обрабатывающих открытый файл.

Функция `IS_OPEN` принимает в качестве параметра описатель файла. Функция `IS_OPEN` проверяет, указывает ли описатель на открытый файл. Она возвращает логическое значение `TRUE`, когда файл уже открыт; в противном случае возвращается логическое значение `FALSE`, показывающее, что файл еще не открыт. Пример на слайде демонстрирует, как совместно используются две подпрограммы.

Примечание: полный синтаксис см. в *PL/SQL Packages and Types Reference 10g*.

Пример использования пакета UTL_FILE

```
CREATE OR REPLACE PROCEDURE sal_status(
  dir IN VARCHAR2, filename IN VARCHAR2) IS
  file UTL_FILE.FILE_TYPE;
 CURSOR empc IS
  SELECT last_name, salary, department_id
  FROM employees ORDER BY department_id;
 newdeptno employees.department_id%TYPE;
 olddeptno employees.department_id%TYPE := 0;
BEGIN
  file:= UTL_FILE.FOPEN (dir, filename, 'w');
  UTL_FILE.PUT_LINE(file,
    'REPORT: GENERATED ON ' || SYSDATE);
  UTL_FILE.NEW_LINE (file); ...
```

Copyright © 2004, Oracle. All rights reserved.

Пример использования пакета UTL_FILE

Процедура sal_status создает отчет о зарплате служащих каждого отдела. Эти данные выводятся в текстовый файл с помощью пакета UTL_FILE. Переменная file объявляется с типом UTL_FILE.FILE_TYPE. Этот тип описывает запись, содержащую три поля:.

```
TYPE file_type IS RECORD (id BINARY_INTEGER, datatype
  BINARY_INTEGER, byte_mode BOOLEAN);
```

Содержимое переменной с типом file_type принадлежит пакету UTL_FILE. Пользователи не должны ссылаться на компоненты этой записи, а также не должны их изменять.

Процедура sal_status принимает два параметра:

- dir – имя директории, в которую пишется текстовый файл;
- filename – имя файла.

Пример вызова процедуры:

```
EXECUTE sal_status('MY_DIR', 'salreport.txt')
```

Примечание. Месторасположение директории (MY_DIR) указывается заглавными буквами, псевдоним директории был создан по команде CREATE DIRECTORY. В случае чтения файла в цикле, выход из него производится при возникновении исключения NO_DATA_FOUND. Пакет UTL_FILE передает выходные данные синхронно. Процедура DBMS_OUTPUT не выдает выходные результаты, пока не завершится процедура.

Пример использования пакета UTL_FILE

```
  FOR emp_rec IN empc LOOP
    IF emp_rec.department_id <> olddeptno THEN
      UTL_FILE.PUT_LINE (file,
       'DEPARTMENT: ' || emp_rec.department_id);
    END IF;
    UTL_FILE.PUT_LINE (file,
       '  EMPLOYEE: ' || emp_rec.last_name ||
       ' earns: ' || emp_rec.salary);
    olddeptno := emp_rec.department_id;
  END LOOP;
  UTL_FILE.PUT_LINE(file,'*** END OF REPORT ***');
  UTL_FILE.FCLOSE (file);
EXCEPTION
 WHEN UTL_FILE.INVALID_FILEHANDLE THEN
  RAISE_APPLICATION_ERROR(-20001,'Invalid File.');
 WHEN UTL_FILE.WRITE_ERROR THEN
  RAISE_APPLICATION_ERROR (-20002, 'Unable to
write to file');
END sal_status;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

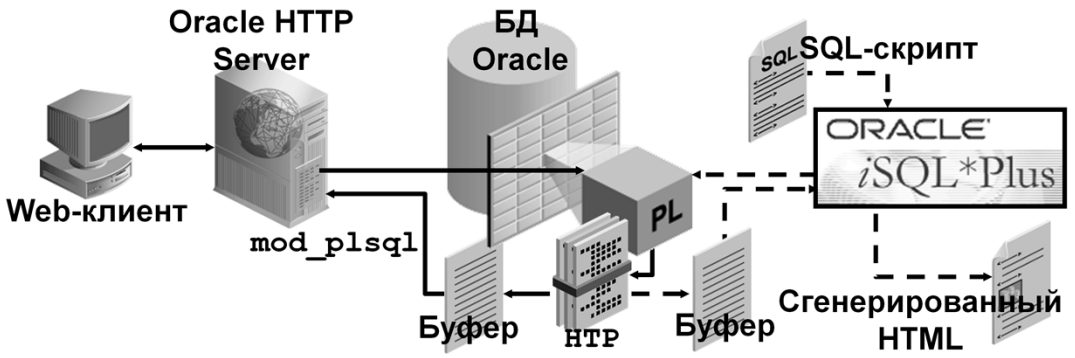
Пример использования пакета UTL_FILE (продолжение)

Выходные данные этого отчета, полученного в файле salreport.txt, имеют вид:

```
SALARY REPORT: GENERATED ON 08-MAR-01
      DEPARTMENT: 10
            EMPLOYEE: Whalen earns: 4400
      DEPARTMENT: 20
            EMPLOYEE: Hartstein earns: 13000
            EMPLOYEE: Fay earns: 6000
      DEPARTMENT: 30
            EMPLOYEE: Raphaely earns: 11000
            EMPLOYEE: Khoo earns: 3100
            ...
      DEPARTMENT: 100
            EMPLOYEE: Greenberg earns: 12000
          ...
      DEPARTMENT: 110
            EMPLOYEE: Higgins earns: 12000
            EMPLOYEE: Gietz earns: 8300
            EMPLOYEE: Grant earns: 7000
      *** END OF REPORT ***
```

Генерация Web-страниц с помощью пакета HTP

- **Процедуры пакета HTP генерируют теги HTML.**
- **Пакет HTP используется для динамической генерации документов на HTML и может быть вызван из:**
  - **браузера с использованием Oracle HTTP Server и служб PL/SQL Gateway (mod_plsql)**
  - **скрипта в *i*SQL*Plus, созданного для вывода результатов на HTML**



Copyright © 2004, Oracle. All rights reserved.

Генерация Web-страниц с помощью пакета HTP

Пакет HTP содержит процедуры для генерации тегов HTML. Обычно данные, предоставляемые различным процедурам в качестве параметров, заключаются в генерируемыми теги HTML. Слайд иллюстрирует два пути использования HTP.

- Вероятнее всего, процедуры будут вызываться с помощью служб PL/SQL Gateway через компонент mod_plsql, поставляемый вместе с Oracle HTTP Server, который входит в Oracle Application Server (сплошная линия на графике).
- Альтернативный вариант (пунктирная линия на графике) предполагает вызов ваших процедур из *i*SQL*Plus, который выводит сгенерированные результаты на HTML. Их можно скопировать (copy) и вставить (paste) в файл. Такой метод применяется в этом курсе, поскольку не установлен Oracle Application Server.

Примечание. Процедуры пакета HTP выводят информацию в буфер сеанса, хранимый на сервере базы данных. В контексте Oracle HTTP, когда процедура завершается, компонент mod_plsql автоматически получает содержимое буфера, которое затем возвращается браузеру в качестве ответа сервера HTTP. В SQL*Plus необходимо вручную выполнить:

- команду SET SERVEROUTPUT ON;
- процедуру, генерирующую код HTML в буфере;
- процедуру OWA_UTIL.SHOWPAGE для вывода содержимого буфера.

Использование процедур пакета HTP

- **Генерация одного или нескольких тегов HTML. Пример:**

```
htp.bold('Hello');              -- <B>Hello</B>
htp.print('Hi <B>World</B>');   -- Hi <B>World</B>
```

- **Создание на HTML отформатированного документа:**

```
BEGIN                           -- Generates:
 htp.htmlOpen;    --------->
 htp.headOpen;    --------->    <HTML>
 htp.title('Welcome');  -->     <HEAD>
 htp.headClose;   --------->    <TITLE>Welcome</TITLE>
 htp.bodyOpen;    --------->    </HEAD>
 htp.print('My home page');      <BODY>
 htp.bodyClose;   --------->    My home page
 htp.htmlClose;   --------->    </BODY>
END;                            </HTML>
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование процедур пакета HTP

Пакет HTP выстроен так, чтобы было однозначное соответствие между входящей в него процедурой и стандартными тегами HTML. Например, для вывода текста жирным шрифтом на Web-странице этот текст должен быть заключен в пару тегов HTML: <B> и </B>. В первом примере кода на слайде показывается, как с помощью соответствующей процедуры BOLD пакета HTP вывести на HTML жирным шрифтом слово Hello. Процедура HTP.BOLD принимает текст в качестве параметра и заключает его в соответствующие теги HTML в генерируемом выходном коде.

Процедура HTP.PRINT копирует текстовый параметр в буфер. Пример на слайде показывает, что передаваемый процедуре HTP.PRINT параметр может содержать теги HTML. Такой способ рекомендуется использовать, когда нельзя сгенерировать теги HTML с помощью процедур пакета HTP.

Второй пример на слайде представляет собой блок PL/SQL, генерирующий документ HTML стандартного вида. Пример приводится для иллюстрации того, как каждая из процедур генерирует соответствующую строку на HTML, показываемую справа в текстовом окне.

Преимущество использования пакета HTP в том, что он создает отформатированные HTML-документы, устраняя необходимость вручную заключать данных в теги HTML.

Примечание: дополнительную информацию о всех процедурах пакета HTP см. в документе *PL/SQL Packages and Types Reference 10g*.

Создание файла HTML в *i*SQL*Plus

Для создания файла HTML в среде *i*SQL*Plus выполните следующие шаги:

1. **Создайте SQL-скрипт, содержащий команды:**

```
SET SERVEROUTPUT ON
ACCEPT procname PROMPT "Procedure: "
EXECUTE &procname
EXECUTE owa_util.showpage
UNDEFINE proc
```

2. **Загрузите и выполните скрипт в *i*SQL*Plus (предоставьте значения для переменных подстановки).**
3. **Выделите, скопируйте и вставьте сгенерированный в браузере текст на HTML в HTML-файл.**
4. **Откройте HTML-файл в браузере.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание файла HTML в *i*SQL*Plus

Пример на слайде демонстрирует шаги генерации с помощью какой-либо процедуры кода HTML и его сохранения в HTML-файле.

1. Включите выдачу результатов на сервере по команде `SET SERVEROUTPUT ON`. Без этого будут получены сообщения о возникших исключениях при выполнении процедур, вызывающих пакет `HTP`.
2. Выполните процедуру, содержащую вызовы пакета `HTP`.
   Примечание: если в процедуре не вызывается пакет `DBMS_OUTPUT`, никакие выходные данные *не* выдаются.
3. Выполните процедуру `OWA_UTIL.SHOWPAGE`, чтобы вывести текст. Фактически будет выдан код на HTML, генерируемый из данных буфера.

Команда `ACCEPT` запрашивает имя выполняемой процедуры. В результате выполнения процедуры `OWA_UTIL.SHOWPAGE` в окне браузера выводятся теги HTML. После этого можно скопировать из окна браузера сгенерированные теги HTML и вставить их в HTML-файл, обычно имеющий расширение `.htm` или `.html`.

Примечание. В SQL*Plus можно воспользоваться командой `SPOOL`, чтобы направить выходной текст на HTML непосредственно в HTML-файл. Команда `SPOOL` не поддерживается в *i*SQL*Plus; единственная возможность – использовать способ копирование-вставка (copy-and-paste).

Использование пакета UTL_MAIL

Пакет UTL_MAIL:

- **Утилита предоставляющая обычно используемые функции управления электронной почтой: вложения, CC, BCC и подтверждение получения.**
- **Необходимо задать параметр инициализации базы данных SMTP_OUT_SERVER**
- **Предоставляет следующие процедуры:**
  - **SEND для передачи сообщений без вложений**
  - **SEND_ATTACH_RAW для передачи сообщений с двоичными вложениями**
  - **SEND_ATTACH_VARCHAR2 для передачи сообщений с текстовыми вложениями**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование пакета UTL_MAIL

Пакет UTL_MAIL – это утилита управления электронной почтой, включающая наиболее часто используемые возможности, например, *вложения* (*attachments*), CC (дополнительные адреса получателей), BCC (адреса скрытых получателей) и подтверждение получения.

По умолчанию пакет UTL_MAIL не инсталлируется, поскольку требуется сконфигурировать параметр инициализации SMTP_OUT_SERVER, а это открывает некоторые сведения, касающиеся безопасности. При инсталляции следует выполнить шаги, предотвращающие перегруженность порта, определенного в SMTP_OUT_SERVER, вызванного передачей данных. Чтобы инсталлировать UTL_MAIL, соединитесь в *i*SQL*Plus как SYSDBA и выполните следующие скрипты:

```
@$ORACLE_HOME/rdbms/admin/utlmail.sql
@$ORACLE_HOME/rdbms/admin/prvtmail.plb
```

Следует также определить параметр SMTP_OUT_SERVER в файле init.ora:

```
SMTP_OUT_SERVER=star.fors.com
```

В параметре SMTP_OUT_SERVER задается SMTP-хост и порт, по которому пакет UTL_MAIL доставляет отправляемые почтовые сообщения. В параметре могут быть заданы через запятую несколько серверов. Если первый сервер в списке недоступен, пакет UTL_MAIL пытается использовать второй сервер и т.д.. Если не задан параметр SMTP_OUT_SERVER, тогда по умолчанию действует установка, получаемая из параметра инициализации DB_DOMAIN. Этот параметр задает логическое расположение базы данных в сетевой структуре. Пример:

```
db_domain=fors.com
```

Инсталляция и использование пакета UTL_MAIL

- Как SYSDBA в *i*SQL*Plus:
  - Задайте параметр SMTP_OUT_SERVER (требуется перезапуск СУБД).

```
ALTER SYSTEM SET SMTP_OUT_SERVER='star.fors.com'
SCOPE=SPFILE
```

  - Инсталлируйте пакет UTL_MAIL.

```
@?/rdbms/admin/utlmail.sql
@?/rdbms/admin/prvtmail.plb
```

- Как разработчик вызовите процедуру UTL_MAIL:

```
BEGIN
 UTL_MAIL.SEND('otn@fors.com','user@fors.com',
   message => 'For latest downloads visit OTN',
   subject => 'OTN Newsletter');
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Инсталляция и использование пакета UTL_MAIL

На слайде показано, как сконфигурировать параметр SMTP_OUT_SERVER и указать в нем имя SMTP-хоста в вашей сети, а также, как проинсталлировать пакет UTL_MAIL, который по умолчанию не установлен. После изменения параметра SMTP_OUT_SERVER требуется перезапустить экземпляр базы данных. Эти действия выполняет пользователь с привилегиями SYSDBA.

В последнем примере показывается самый простой способ передачи текстового сообщения, содержащего тему (subject) и само сообщение (message), с помощью процедуры UTL_MAIL.SEND. Первые два параметра следующие:

- sender (в нашем примере адрес отправителя – otn@fors.com)
- recipients (в нашем примере адрес получателя – user@fors.com). Значение может представлять собой список адресов, разделенных запятыми.

Процедура UTL_MAIL.SEND предоставляет и некоторые другие параметры, например, cc, bcc и priority, для которых определены значения по умолчанию . В примере в message задан текст сообщения, а в subject – тема. Чтобы послать HTML-сообщение с тегами HTML, добавьте параметр mime_type (например, mime_type=>'text/html').

Примечание: подробные сведения о всех параметрах процедур пакета UTL_MAIL см. в документе *PL/SQL Packages and Types Reference*.

Передача сообщения с двоичным вложением по электронной почте

Используйте процедуру UTL_MAIL.SEND_ATTACH_RAW:

```
CREATE OR REPLACE PROCEDURE send_mail_logo IS
BEGIN
  UTL_MAIL.SEND_ATTACH_RAW(
    sender => 'me@fors.com',
    recipients => 'you@somewhere.net',
    message =>
      '<HTML><BODY>See attachment</BODY></HTML>',
    subject => 'Oracle Logo',
    mime_type => 'text/html'
    attachment => get_image('oracle.gif'),
    att_inline => true,
    att_mime_type => 'image/gif',
    att_filename => 'oralogo.gif');
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Передача сообщения с двоичным вложением по электронной почте

На слайде показана процедура UTL_MAIL.SEND_ATTACH_RAW, предназначенная для посылки текстового сообщения или HTML-сообщения с двоичным вложением. Кроме параметров sender, recipients, message, subject и mime_type, предоставляющих значения для основной части почтового сообщения, процедура SEND_ATTACH_RAW имеет следующие выделенные на слайде параметры:

- attachment (обязательный параметр) принимает данное типа RAW с максимальным размером 32767 знаков в двоичном представлении.
- att_inline (необязательный параметр) с логическим типом данных (значение по умолчанию TRUE), отражающим, доступно ли для просмотра вложение вместе с телом сообщения.
- att_mine_type (необязательный параметр) задает формат вложения. Если параметр не задан, его значение по умолчанию – application/octet.
- att_filename (необязательный параметр) присваивает имя файла для вложения. По умолчанию значение – NULL и тогда передаваемому файлу имя не присваивается.

Передача сообщения с двоичным вложением по электронной почте (продолжение)

В примере используется функция get_image для чтения данных изображения из BFILE . Для чтения данных с типом BFILE требуется создать в базе данных логическое имя директории, используя для этого команду CREATE DIRECTORY.Подробна обработка данных типа BFILE рассматривается в уроке “Работа с большими объектами”. Код функции get_image показан ниже.

В функции get_image используется пакет DBMS_LOB для чтения двоичного файла из операционной системы:

```
CREATE OR REPLACE FUNCTION get_image(
     filename VARCHAR2, dir VARCHAR2 := 'TEMP')
RETURN RAW IS
  image RAW(32767);
  file  BFILE := BFILENAME(dir, filename);
BEGIN
  DBMS_LOB.FILEOPEN(file, DBMS_LOB.FILE_READONLY);
  image := DBMS_LOB.SUBSTR(file);
  DBMS_LOB.CLOSE(file);
  RETURN image;
END;
/
```

Чтобы создать в БД директорию TEMP, выполните в *i*SQL*Plus следующую команду:

```
CREATE DIRECTORY temp AS 'c:\temp';
```

Примечание: для выполнения этой команды требуется системная привилегия CREATE ANY DIRECTORY.

Передача сообщения с текстовым вложением по электронной почте

Используйте процедуру UTL_MAIL.SEND_ATTACH_VARCHAR2:

```
CREATE OR REPLACE PROCEDURE send_mail_file IS
BEGIN
  UTL_MAIL.SEND_ATTACH_VARCHAR2(
    sender => 'me@fors.com',
    recipients => 'you@somewhere.net',
    message =>
      '<HTML><BODY>See attachment</BODY></HTML>',
    subject => 'Oracle Notes',
    mime_type => 'text/html'
    attachment => get_file('notes.txt'),
    att_inline => false,
    att_mime_type => 'text/plain',
    att_filename => 'notes.txt');
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Передача сообщения с текстовым вложением по электронной почте

На слайде показана процедура UTL_MAIL.SEND_ATTACH_VARCHAR2, предназначенная для посылки текстового сообщения или HTML-сообщения с текстовым вложением. Кроме параметров sender, recipients, message, subject и mime_type, предоставляющих значения для основной части почтового сообщения, процедура SEND_ATTACH_VARCHAR2 имеет следующие выделенные на слайде параметры:

- attachment (обязательный параметр) принимает данное типа VARCHAR2 с максимальным размером 32767 знаков в двоичном представлении.
- att_inline (необязательный параметр) с логическим типом данных (значение по умолчанию TRUE), отражающим, доступно ли для просмотра вложение вместе с телом сообщения.
- att_mine_type (необязательный параметр) задает формат вложения. Если параметр не задан, его значение по умолчанию – application/octet.
- att_filename (необязательный параметр) присваивает имя файла для вложения. По умолчанию значение – NULL и тогда передаваемому файлу имя не присваивается.

В примере на слайде для задания значение параметра attachment применяется функция get_file, которая читает текстовый файл (используя тип BFILE) из директории ОС. Для задания этого параметра можно также воспользоваться переменной типа VARCHAR2. Код функции приведен на следующей странице.

Передача сообщения с текстовым вложением по электронной почте (продолжение)

Функция get_file использует пакет DBMS_LOB для чтения двоичного файла из операционной системы и пакет UTL_RAW для преобразования двоичных данных с типом RAW в читабельный текст в виде, предоставляемом типом данных VARCHAR2:

```
CREATE OR REPLACE FUNCTION get_file(
  filename VARCHAR2, dir VARCHAR2 := 'TEMP')
RETURN VARCHAR2 IS
  contents VARCHAR2(32767);
  file  BFILE := BFILENAME(dir, filename);
BEGIN
  DBMS_LOB.FILEOPEN(file, DBMS_LOB.FILE_READONLY);
  contents := UTL_RAW.CAST_TO_VARCHAR2(
                  DBMS_LOB.SUBSTR(file));
  DBMS_LOB.CLOSE(file);
  RETURN contents;
END;
/
```

Примечание: вы можете воспользоваться другим подходом и с помощью функциональных возможностей пакета UTL_FILE прочитать содержимое текстового файла в переменную типа VARCHAR2.

Для выполнения предыдущего примера требуется создать директорию. Например, это можно сделать в *i*SQL*Plus по следующей команде:

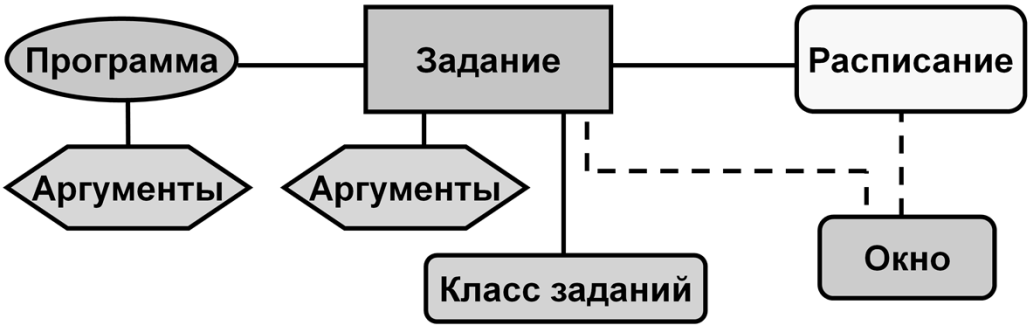
```
CREATE DIRECTORY temp AS 'c:\temp';
```

Примечание: для выполнения этой команды требуется системная привилегия CREATE ANY DIRECTORY.

Пакет DBMS_SCHEDULER

Для выполнения задания используются компоненты планировщика базы данных. Задание создается с помощью пакета DBMS_SCHEDULER:

- **Уникальное имя задания**
- **Программа (“что” должно быть выполнено)**
- **Расписание (“когда” должно быть выполнено)**



Copyright © 2004, Oracle. All rights reserved.

Пакет DBMS_SCHEDULER

База данный Oracle 10*g* предоставляет подпрограммы пакета DBMS_SCHEDULER, упрощающие управление и имеющие широкие возможности для решения сложных задач планирования. Совокупность подпрограмм этого пакета называют планировщиком (Scheduler) и их можно вызвать из любой программы PL/SQL. Планировщик позволяет администраторам БД и разработчикам приложений управлять временем (“когда”) и местом (“где”) выполнения различных задач. Возможность выполнение многих рутинных задач базы данных без ручного вмешательства снижает текущие расходы, позволяет создавать более надежный код и минимизирует ошибки человека.

На диаграмме показаны следующие компоненты архитектуры планировщика.

- **Задание (job)** объединяет *программу* (*program*) и *расписание* (*schedule*). Аргументы, требуемые программе, могут быть предоставлены программе или заданию. Все имена заданий имеют вид: [схема.]имя. При создании задания задается его имя, программа, расписание и (необязательно) характеристики, определяемые через **класс заданий (job class)**.
- **Программа (program)** определяет, что должно быть выполнено. Каждое автоматическое задание вызывает исполняемый модуль, который может быть блоком PL/SQL, хранимой процедурой, двоичным модулем во внутреннем коде (native binary executable) или скриптом оболочки (shell script). Программа предоставляет метаданные о конкретном исполняемом модуле и для нее может быть задан список аргументов.

Пакет `DBMS_SCHEDULER` (продолжение)

- **Расписание** (**schedule**) определяет, когда и как часто выполняется задание.
- **Класс заданий** (**job class**) определяет категорию заданий с общими требованиями по использованию ресурсов и другими общими характеристиками. В любой момент времени каждое задание принадлежит только одному классу заданий. Класс заданий имеет следующие атрибуты:
  - имя **сервиса** базы данных (database **service** name); задания класса привязываются к определенному сервису, в результате чего они выполняются экземпляром, обслуживающим данный сервис;
  - **группа потребителей ресурсов** (**resource consumer group)** выделяет множество пользовательских сеансов с общими ресурсными требования, необходимыми для обработки; в каждый определенный момент времени пользовательский сеанс или класс заданий принадлежит единственной группе потребителей ресурсов; в соответствие с тем, с какой группой потребителей связан класс заданий, ему выделяются ресурсы.
- **Окно (window)** отражает интервал времени с четко очерченными границами начала и завершения. Оно используется для активизации различных ресурсных планов в различное время.

На слайде компоненты задания представлены как основополагающие сущности, образующие задание. Однако программа, расписание, окно и класс заданий могут быть созданы как отдельные сущности, а затем привязаны к заданию, выполняемому планировщиком. При создании задания все его компоненты могут быть определены непосредственно (in-line) в коде команды. Альтернативный подход – создание задания, содержащего ссылки на программу и расписание, которые были до этого определены. Примеры создания заданий будут рассмотрены далее в этом уроке.

Дополнительные сведения о планировщике можно получить в электронном курсе (eStudy) *Oracle Database 10g: Configure and Manage Jobs with the Scheduler*.

Создание задания

Задание можно создать несколькими путями, комбинируя явно заданные параметры, имена программы и расписания. Процедура `CREATE_JOB` создает задание, используя:

- **Явно (in-line) заданную в параметрах информацию о том, “что” выполнять и по какому расписанию**
- **Имя хранимой программы и явно заданное расписание**
- **Непосредственное определение задачи и использование сохраненного расписания**
- **Хранимую программу и хранимое расписание**

Примечание: для создания задания необходима системная привилегия `CREATE JOB`.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание задания

Компонент, вызывающий выполнение чего-либо в заданное время, называется **заданием (job)**. Процедура `DBMS_SCHEDULER.CREATE_JOB` создает задание в отключенном по умолчанию состоянии. Задание переводится в активное и планируемое для выполнения состояние после его явного включения. При создании задания:

- задается имя в виде: `[схема.]имя`
- необходима привилегия `CREATE JOB`.

Примечание. Пользователь с привилегией `CREATE ANY JOB` может создать задание в любой схеме, кроме `SYS`. Для того, чтобы связать задание с определенным классом, необходима привилегия `EXECUTE` на этот класс.

В простом варианте задание можно создать, задавая в аргументах процедуры `CREATE_JOB` всю информацию: программу, которая должна быть выполнена (что), и по какому расписанию (когда). В альтернативном варианте используются предопределенные компоненты: программа и расписание. Если такие компоненты есть, тогда их имена можно указывать при создании задания, а также комбинировать с непосредственно (in-line) задаваемыми аргументами. Такой подход предоставляет максимальную гибкость при создании задания.

Когда задание создается, производится простая логическая проверка информации о расписании. База данных проверяет, следует ли конечная дата за датой начала. Если дата начала указывает на момент в прошлом, тогда она изменяется на текущую дату.

Создание задания с явно задаваемыми параметрами

Укажите в аргументах процедуры CREATE_JOB тип кода, сам код, дату начала и частоту выполнения задания.

Пример планирования ежечасного выполнения блока PL/SQL:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB(
   job_name => 'JOB_NAME',
   job_type => 'PLSQL_BLOCK',
   job_action => 'BEGIN ...; END;',
   start_date => SYSTIMESTAMP,
   repeat_interval=>'FREQUENCY=HOURLY;INTERVAL=1',
   enabled => TRUE);
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание задания с явно задаваемыми параметрами

Используя процедуру DBMS_SCHEDULER.CREATE_JOB, можно создать задание для выполнения блока PL/SQL, хранимой процедуры или внешней программы. Процедурой CREATE_JOB можно воспользоваться непосредственно, не создавая заранее такие компоненты, как программа и расписание.

В примере на слайде показывается, как можно непосредственно в командной строке (in-line) задать всю информацию о создаваемом задании. В параметрах процедуры CREATE_JOB определяется, “что” должно быть выполнено, по какому расписанию и другие атрибуты задания. Следующие параметры определяют то, что должно быть выполнено:

- job_type (тип задания) может иметь одно из трех значений:
  - PLSQL_BLOCK для блока PL/SQL или команды SQL; задание такого типа не может иметь аргументов;
  - STORED_PROCEDURE для хранимой автономной или пакетной процедуры; процедуры могут принимать значения аргументов, передаваемых заданию;
  - EXECUTABLE для приложения, запускаемого из командной строки ОС.
- расписание задается с помощью следующих параметров:
  - start_date (дата начала) в формате даты и времени (time stamp), а также repeat_interval (интервал повторения) как календарное выражение или PL/SQL-выражение; можно также указать end_date (дату окончания).

Примечание. Строковые выражения, задаваемые в параметре repeat_interval, рассматриваются далее в этом уроке. В примере определяется, что задание должно выполняться каждый час.

Использование программы в создаваемом задании

- **Создайте программу с помощью процедуры `CREATE_PROGRAM`:**

```
BEGIN
  DBMS_SCHEDULER.CREATE_PROGRAM(
   program_name => 'PROG_NAME',
   program_type => 'PLSQL_BLOCK',
   program_action => 'BEGIN ...; END;');
END;
```

- **Воспользуйтесь перезагружаемой процедурой `CREATE_JOB` с параметром `program_name`:**

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
   program_name => 'PROG_NAME',
   start_date => SYSTIMESTAMP,
   repeat_interval => 'FREQ=DAILY',
   enabled => TRUE);
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование программы в создаваемом задании

С помощью процедуры `DBMS_SCHEDULER.CREATE_PROGRAM` определяется программа, которой должно быть присвоено уникальное имя. Создание программы отдельно от задания позволяет:

- один раз определить действие, которым затем можно многократно воспользоваться во многих заданиях;
- изменить расписание задания без пересоздания блока PL/SQL;
- изменить исполняемую программу без изменения всех заданий.

Строка в параметре `program_action` задает имя процедуры, исполняемого модуля или блок PL/SQL в зависимости от значения параметра `program_type`, которое может быть определено как:

- `PLSQL_BLOCK` (анонимный блок или команда SQL);
- `STORED_PROCEDURE` (хранимая процедура, например на PL/SQL, Java или C);
- `EXECUTABLE` (команда, вызываемая для исполнения из командной строки ОС).

В примере на слайде демонстрируется вызов анонимного блока PL/SQL. В следующем примере показывается вызов внешней процедуры из программы:

```
DBMS_SCHEDULER.CREATE_PROGRAM(program_name => 'GET_DATE',
    program_action => '/usr/local/bin/date',
    program_type => 'EXECUTABLE');
```

Для создания задания, в котором используется программа, применяется процедура `DBMS_SCHEDULER.CREATE_JOB`, в вызове которой задается имя программы в параметре `program_name`.

Использование программы с аргументами в создаваемом задании

- **Создание программы:**

```
DBMS_SCHEDULER.CREATE_PROGRAM(
  program_name => 'PROG_NAME',
  program_type => 'STORED_PROCEDURE',
  program_action => 'EMP_REPORT');
```

- **Определение аргумента:**

```
DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT(
  program_name => 'PROG_NAME',
  argument_name => 'DEPT_ID',
  argument_position=> 1, argument_type=> 'NUMBER',
  default_value => '50');
```

- **Создание задания, для которого определяется количество аргументов:**

```
DBMS_SCHEDULER.CREATE_JOB('JOB_NAME', program_name
  => 'PROG_NAME', start_date => SYSTIMESTAMP,
  repeat_interval => 'FREQ=DAILY',
  number_of_arguments => 1, enabled => TRUE);
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование программы с аргументами в создаваемом задании

Таким программам, как процедуры PL/SQL или внешние процедуры, могут потребоваться входные аргументы. Для существующей программы аргумент можно определить с помощью процедуры `DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT`. Процедура `DEFINE_PROGRAM_ARGUMENT` имеет следующие параметры:

- `program_name`; задает существующую программу, которая должна быть изменена;.
- `argument_name`; задает уникальное имя для аргумента программы;
- `argument_position`; задает позицию, в которой аргумент передается при вызове программы;
- `argument_type`; задает тип данных значения аргумента, передаваемого вызываемой программе;
- `default_value`; значение по умолчанию, предоставляемое программе, когда задание, в котором запланировано выполнение программы, не предоставляет значения.

На слайде показывается, как создать задание, выполняющее программу с одним аргументом. По умолчанию значение аргумента программы – 50. Чтобы изменить значение аргумента программы, воспользуйтесь следующей процедурой:

```
DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE(
  job_name => 'JOB_NAME',
  argument_name => 'DEPT_ID', argument_value => '80');
```

Использование расписания в создаваемом задании

- **Создайте расписание с помощью процедуры `CREATE_SCHEDULE`:**

```
BEGIN
  DBMS_SCHEDULER.CREATE_SCHEDULE('SCHED_NAME',
   start_date => SYSTIMESTAMP,
   repeat_interval => 'FREQ=DAILY',
   end_date => SYSTIMESTAMP +15);
END;
```

- **В вызове процедуры `CREATE_JOB` укажите в параметре `schedule_name` ссылку на расписание:**

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
   schedule_name => 'SCHED_NAME',
   job_type => 'PLSQL_BLOCK',
   job_action => 'BEGIN ...; END;',
   enabled => TRUE);
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование расписания в создаваемом задании

Можно создать общее расписание, применяемое для различных заданий. В этом случае не надо каждый раз при создании задания указывать подробную информацию о расписании. Преимущества создания расписания:

- повторная используемость и возможность его назначения различным заданиям;
- изменение расписания влияет на все задания, использующие это расписание; в таком случае расписание изменяется один раз, а не многократно (в каждом задании).

Расписание задается с точностью до ближайшей секунды. Хотя тип данных `TIMESTAMP` предоставляет данные с большей точностью, планировщик округляет любое значение до ближайшей секунды.

Время начала и окончания расписания задается как тип данных `TIMESTAMP`. Параметр `end_date` для хранимого расписания определяет дату, после которой расписание больше не действует. В примере расписание действует 15 дней после начала использования задания.

Параметр `repeat_interval` (интервал повторения) для хранимого расписания должен быть задан в виде календарного выражения. Неопределенное (`NULL`) значение параметра `repeat_interval` означает, что задание выполняется только один раз.

Примечание: в хранимом расписании нельзя использовать PL/SQL-выражения для указания интервала повторения.

Установка интервала повторения задания

- **Использование календарного выражения:**

```
repeat_interval=> 'FREQ=HOURLY; INTERVAL=4'
repeat_interval=> 'FREQ=DAILY'
repeat_interval=> 'FREQ=MINUTELY;INTERVAL=15'
repeat_interval=> 'FREQ=YEARLY;
                   BYMONTH=MAR,JUN,SEP,DEC;
                   BYMONTHDAY=15'
```

- **Использование PL/SQL-выражения:**

```
repeat_interval=> 'SYSDATE + 36/24'
repeat_interval=> 'SYSDATE + 1'
repeat_interval=> 'SYSDATE + 15/(24*60)'
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Установка интервала повторения задания

Для планирования интервала повторения задания можно воспользоваться PL/SQL-выражением (задавая его только в качестве аргумента задания) или календарным выражением.

Примеры на слайде:

- `FREQ=HOURLY;INTERVAL=4` – повторение через каждые четыре часа;
- `FREQ=DAILY` – ежедневное повторение в одно и тоже время, совпадающее с временем начала действия расписания;
- `FREQ=MINUTELY;INTERVAL=15` – повторение через каждые 15 минут;.
- `FREQ=YEARLY;BYMONTH=MAR,JUN,SEP,DEC;BYMONTHDAY=15` – ежегодное повторение в следующие дни: 15 марта, 15 июня, 15 сентября и 15 декабря.

При использовании календарного выражения время следующего запуска задания вычисляется с помощью интервала повторения и даты начала планирования задания (`start_date`).

Примечание: если интервал повторения не задан (иными словами значение аргумента – `NULL`), задание выполняется только один раз в указанное в параметре `start_date` время.

Создание задания, в котором используются программа и расписание с именами

- Создайте программу PROG_NAME с помощью процедуры CREATE_PROGRAM.
- Создайте расписание SCHED_NAME с помощью процедуры CREATE_SCHEDULE.
- Создайте задание, в котором указываются ссылки на программу и расписание :

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
   program_name => 'PROG_NAME',
   schedule_name => 'SCHED_NAME',
   enabled => TRUE);
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание задания, в котором используются программа и расписание с именами

В примере на слайде показан последний вид использования процедуры DBMS_SCHEDULER.CREATE_JOB. В вызове этой процедуры в соответствующих параметрах указываются имена программы (PROG_NAME) и расписания (SCHED_NAME).

На этом примере видно, как просто может быть создано задание, когда используются предопределенная программа и расписание.

В этом курсе не рассматриваются слишком сложные планы и расписания. Например, можно создать окно для повторяющихся во времени планов и связать ресурсный план с окном. Ресурсный план будет определять атрибуты использования ресурсов, требуемых в период, определяемый выполняемым (активным) окном.

Управление заданиями

- **Выполнение задания:**

```
DBMS_SCHEDULER.RUN_JOB('SCHEMA.JOB_NAME');
```

- **Остановка задания:**

```
DBMS_SCHEDULER.STOP_JOB('SCHEMA.JOB_NAME');
```

- **Удаление задания, даже если оно в настоящий момент выполняется:**

```
DBMS_SCHEDULER.DROP_JOB('JOB_NAME', TRUE);
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Управление заданиями

После того, как задание создано, вы можете:

- Выполнить задание. Для этого вызывается процедура `RUN_JOB`, которой передается имя задания, немедленно выполняемого в текущем сеансе.
- Остановить задание с помощью процедуры `STOP_JOB`. Если в текущий момент времени задание выполняется, оно немедленно останавливается. Процедура `STOP_JOB` имеет два аргумента:
  - `job_name`; имя задания, которое необходимо остановить.
  - `force`; заданию передаются сигналы остановиться. Если подчиненный процесс не реагирует и для параметра задано значение `TRUE`, тогда подчиненный процесс, выполняющий задание, останавливается. Для использования возможности принудительной остановки задания (`force`) требуется системная привилегия `MANAGE SCHEDULER`.
- Удалить задание с помощью процедуры `DROP_JOB`. Эта процедура имеет два аргумента:
  - `job_name`; имя задания, которое необходимо удалить.
  - `force`; должно ли задание быть остановлено и удалено, если оно в текущий момент выполняется (значение по умолчанию – `FALSE`.)

  Когда при вызове процедуры `DROP_JOB` выполняется задание, которое необходимо удалить, вызов завершается аварийно, если только для параметра `force` не задано значение `TRUE`. Если же задано значения `TRUE`, тогда любой выполняемый экземпляр задания останавливается и задание удаляется.

Примечание: для выполнения, остановки или удаления задания, принадлежащего другому пользователю, требуются привилегии `ALTER` и `EXECUTE` на это задание или системные привилегии `CREATE ANY JOB` и `EXECUTE ANY JOB`.

Представления словаря данных

- [DBA | ALL | USER]_SCHEDULER_JOBS
- [DBA | ALL | USER]_SCHEDULER_RUNNING_JOBS
- [DBA | ALL]_SCHEDULER_JOB_CLASSES
- [DBA | ALL | USER]_SCHEDULER_JOB_LOG
- [DBA | ALL | USER]_SCHEDULER_JOB_RUN_DETAILS
- [DBA | ALL | USER]_SCHEDULER_PROGRAMS

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Представления словаря данных

Представление DBA_SCHEDULER_JOB_LOG показывает все завершившиеся (как удачно, так и с ошибками) экземпляры заданий.

Для просмотра состояния ваших заданий воспользуйтесь следующим запросом:

```
SELECT job_name, program_name, job_type, state
FROM USER_SCHEDULER_JOBS;
```

Чтобы выяснить, каким экземпляром выполняется задание, выполните запрос:

```
SELECT owner, job_name, running_instance,
resource_consumer_group
FROM DBA_SCHEDULER_RUNNING_JOBS;
```

Для получения сведений о том, как выполняется задание, используйте следующий запрос:

```
SELECT job_name, instance_id, req_start_date,
actual_start_date, end_date, completion_status
FROM ALL_SCHEDULER_JOB_LOG;
```

Чтобы выяснить статус ваших заданий, выполните следующий запрос:

```
SELECT job_name, status, error#, run_duration, cpu_used
FROM USER_SCHEDULER_JOB_RUN_DETAILS;
```

Итоги

- Использования различных предварительно инсталлированных пакетов, предоставляемых сервером Oracle
- Использование следующих пакетов:
  - `DBMS_OUTPUT` для помещения в буфер и вывода текста
  - `UTL_FILE` для записи в текстовые файлы операционной системы
  - `HTP` для генерации документов на HTML
  - `UTL_MAIL` для передачи сообщений с вложениями
  - `DBMS_SCHEDULER` для автоматической обработки
- Создание отдельных пакетов и использование скрипта `catproc.sql`

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Итоги

В этом уроке было рассмотрено небольшое подмножество пакетов, предоставляемых базой данных Oracle.

Пакет `DBMS_OUTPUT` используется для целей отладки и для вывода в *i*SQL*Plus данных на экран, генерируемых процедурами.

В этом уроке вы познакомились с возможностями базы данных по созданию файлов в операционной системе (пакет `UTL_FILE`), генерированию отчетов на HTML (пакет `HTP`), передаче электронной почты с и без двоичными или текстовыми вложениями (пакет `UTL_MAIL`), а также планированию выполнения кода PL/SQL и внешнего кода (пакет `DBMS_SCHEDULER`).

Примечание: дополнительную информацию о пакетах и типах PL/SQL см. в документе *PL/SQL Packages and Types Reference 10g*.

Обзор практического занятия 5

- Использование пакета `UTL_FILE` для генерации текстового отчета
- Использование пакета `HTP` для генерации Web-отчета
- Использование пакета `DBMS_SCHEDULER` для автоматической подготовки отчетов

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Обзор практического занятия 5

На этом практическом занятии вы будете использовать пакет `UTL_FILE` для генерации текстового файла с отчетом о сотрудниках всех отделов. Вы создадите процедуру для генерации версии отчета о сотрудниках на HTML и напишите скрипт для сброса (spool) результатов в текстовый файл. Вы будете использовать пакет `DBMS_SCHEDULER` для выполнения отчета, который создает текстовый файл каждые 5 минут.

Практическое занятие 5

1. Создайте процедуру EMPLOYEE_REPORT, которая генерирует отчет о сотрудниках и записывает его в файл операционной системы с помощью пакета UTL_FILE. В отчете должен выдаваться список сотрудников, зарабатывающих выше среднего оклада в их отделе.
   (а). Ваша программа должна принимать два параметра. Первый параметр – директория для вывода, второй – имя записываемого текстового файла.
   Примечание: используйте директорию ОС с именем UTL_FILE. Добавьте секцию обработки исключений для обработки ошибок, которые могут возникнуть при использовании пакета UTL_FILE.
   (б). Вызовите программу, используя для второго параметра имя файла в виде sal_rptxx.txt
   где xx – отображает номер вашего пользователя (например, 01, 15 и т.д.). Ниже приводится пример вывода из файла отчета:

```
Employees who earn more than average salary:
REPORT GENERATED ON 26-FEB-04
Hartstein                    20      $13,000.00
Raphaely                     30      $11,000.00
Marvis                       40      $6,500.00
     ...
     *** END OF REPORT ***
```

   Примечание: в выходных результатах для каждого сотрудника выводится фамилия, номер отдела и оклад. Спросите вашего инструктора, как получить ваш файл с сервера, используя утилиту Putty PSFTP.

2. Создайте новую процедуру WEB_EMPLOYEE_REPORT, которая генерирует те же самые данные, что и процедура EMPLOYEE_REPORT.
   (а). Сначала выполните команду SET SERVEROUTPUT ON, затем процедуру htp.print('hello'), а потом – OWA_UTIL.SHOWPAGE. Выводимые сообщения об исключениях можно проигнорировать.
   (б). Используя пакет HTP, напишите процедуру WEB_EMPLOYEE_REPORT для генерации на HTML отчета о сотрудниках, имеющих оклад, превышающий средний оклад в их отделе. Если вы знакомы с HTML, тогда создайте HTML-таблицу, иначе просто выведите строки данных.
   Подсказка: скопируйте определение курсора и цикл FOR из процедуры EMPLOYEE_REPORT в качестве основы вашего Web-отчета .
   (в). В *i*SQL*Plus вызовите процедуру WEB_EMPLOYEE_REPORT, чтобы сгенерировать данные HTML в серверном буфере, а затем выполните процедуру OWA_UTIL.SHOWPAGE для вывода содержимого буфера на дисплей. Перед выполнением этого кода для параметра SERVEROUTPUT среды *i*SQL*Plus должно быть установлено значение ON.
   (г). Создайте HTML-файл с именем web_employee_report.htm, содержащий текст выходных результатов, которые были вами выбраны. Для этого скопируйте (copy) данные, начиная с открытого тега <HTML> и заканчивая закрытым тегом </HTML> , а затем вставьте (paste) скопированный текст в файл и сохраните его на диске. С помощью двойного щелчка на имени файла выведите его содержимое в браузере по умолчанию.

Практическое занятие 5 (продолжение)

3. Ваш начальник хочет, чтобы отчет о сотрудниках выполнялся часто. Для того чтобы планировать выполнение процедуры `EMPLOYEE_REPORT`, создайте процедуру с использованием пакета `DBMS_SCHEDULER`. В качестве параметров этой процедуре должна передаваться частота получения отчета и необязательно время в минутах, через которое должно быть прекращено запланированное выполнение задания .

   (а). Создайте процедуру `SCHEDULE_REPORT` с двумя следующими параметрами:
   - `interval` для задания строки, отражающей частоту выполнения запланированного задания;
   - `minutes` для задания общего времени жизни в минутах (по умолчанию 10) запланированного задания, после которого оно не должно снова выполняться; в коде при расчете времени остановки необходимо прибавлять к значению текущей даты и времени величину, полученную путем деления передаваемой продолжительности на число, равное (24x60).

   Создайте задание `EMPSAL_REPORT`, используя вызов процедуры `DBMS_SCHEDULER.CREATE_JOB`, в котором должно быть указано, что задание включено и запланировано немедленное выполнение блока PL/SQL. В этом анонимном блоке должна вызываться процедура `EMPLOYEE_REPORT`, которой передается в качестве параметра имя выходного файла, изменяемое в зависимости от времени получения отчета. Процедуре `EMPLOYEE_REPORT` передается имя директории, использовавшейся при решении задания 1, а также имя файла в виде: `sal_rptxx_hh24-mi-ss.txt`, где `xx` – номер используемого вами пользователя и `hh24-mi-ss` – часы, минуты и секунды момента запуска отчета.

   Для конструирования блока PL/SQL используйте следующую локальную переменную PL/SQL:

```
plsql_block VARCHAR2(200) :=
'BEGIN'||
' EMPLOYEE_REPORT(''UTL_FILE'','||
'''sal_rptXX_''||to_char(sysdate,''HH24-MI-
S'')||''.txt'');'||
'END;';
```

   Этот код помогает создать блок PL/SQL, в котором используется приведенная нетривиальная строка PL/SQL. **XX** в коде PL/SQL – это номер используемого вами пользователя.

   (б). Проверьте процедуру `SCHEDULE_REPORT`. Для этого выполните процедуру с параметрами, задающими частоту генерации отчетов через каждые 2 минуты и прекращение повторного получения отчетов через 10 минут после старта задания.
   Примечание: чтобы проверить создание файлов, подсоединитесь к серверу базы данных с помощью утилиты `PSFTP` .

Практическое занятие 5 (продолжение)

(в). Во время выполнения запланированной обработки можно проверять, что задание еще существует. Для этого используйте запрос столбцов `job_name` и `enabled` из представления `USER_SCHEDULER_JOBS`.
Примечание: по истечении 10 минут этот запрос перестанет возвращать строки.